A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1(✉)], Dung Van Bui[2,3], Nhut Tri Do[2,3], and Van Du Nguyen[4]

[1] Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam
thaivn_ph@utc.edu.vn
[2] Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
[3] University of Information Technology, Ho Chi Minh City, Vietnam
dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn
[4] Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam
nvdu@hcmuaf.edu.vn

Abstract Deploying machine learning (ML) based intrusion detection on resource-constrained edge devices requires balancing accuracy, latency, and operational cost. We present a split-deployment IDS architecture across two NVIDIA Jetson Orin Nano Super devices connected through Apache Kafka. The system implements a two-stage pipeline: (1) a lightweight autoencoder anomaly gate on Jetson #1 filters benign traffic, and (2) a PySpark `PipelineModel` classifier on Jetson #2 processes only suspicious flows. Central infrastructure (Kafka, PostgreSQL, InfluxDB, Grafana) runs on a host PC. The classifier is the model selected by our offline Spark ML evaluation, which is leakage-aware (label-leaking port features removed, feature-level duplicate flows eliminated). We evaluate three distributed modes (pipeline split, horizontal Kafka consumer scaling, and a Spark standalone cluster) while streaming CICIDS2017 network flows. Following every flow from send to verdict, queueing included, pipeline split sustains 22 flows/s against 2.5 for a single node running the whole pipeline ($\approx 9\times$), at an end-to-end p95 of about 7 s and 0.60 J per verdict against 3.04 J. Most of the gain comes from decoupling the stages rather than from the second board: the gate run inside the classifier's process gains nothing, while the two stages as separate processes on one board sustain 17.8 flows/s at 0.32 J per verdict. The price is recall. Over all 10,000 replayed flows the deployed pipeline raises no false alarm (precision 100%) but detects 69.5% of attack flows (F1 0.82). Every missed attack was skipped by the gate, and none of the flows it forwarded was misclassified.

Keywords: Intrusion detection · Edge computing · Jetson Orin Nano Super · Kafka · PySpark · Stream processing.

1 Introduction

Network intrusion detection systems (IDS) increasingly must operate closer to data sources to reduce bandwidth and response time [28, 11]. Recent work confirms that

accurate detectors now fit on single-board hardware: ensemble and metaheuristic-optimized detectors have been demonstrated on IoT-edge platforms [6, 21], lightweight models have been designed for deployment on industrial control devices [31], and NVIDIA Jetson-class boards have been used both for autoencoder-based IIoT detection [12] and, on the Jetson Orin Nano specifically, for generative-AI-assisted traffic classification [8]. Such devices nonetheless offer limited CPU, memory, and thermal headroom, so a monolithic detector that classifies every flow saturates the board quickly.

Three gaps persist in this literature. First, the recent edge-IDS studies above evaluate a *single* device: streaming middleware such as Kafka has been paired with Spark for distributed detection [20], but the resulting pipelines are not measured on a multi-node cluster of constrained edge boards with explicit role separation, so it remains unclear how much a second board actually buys and in which distribution mode. Second, energy is now a design constraint in its own right at the edge [22], yet throughput, latency percentiles, and per-inference energy are rarely reported together for one deployed system. Third, offline detection scores are increasingly under scrutiny: methodological pitfalls in security ML [2] and documented labeling and duplication errors in CICIDS2017 [16] mean that a model selected under a leaky protocol may be optimistic by several F1 points before it is ever deployed.

We address all three at once. A practical edge IDS must decide *which* model to deploy (small, accurate, explainable, and selected under a leakage-aware protocol) and *how* to distribute inference across nodes under streaming load. We therefore split detection into a lightweight autoencoder gate and a heavier PySpark classifier, place the two stages in different ways on Jetson Orin Nano Super boards, and measure each placement end to end. The building blocks (an autoencoder, Kafka, Spark MLlib) are established. We do not propose a new detector. The contribution is empirical: which placement pays off on real edge hardware, why, and what the gate costs in detection.

Contributions. (1) A placement study of a two-stage IDS (autoencoder gate, Spark classifier) on a two-board Jetson cluster: single node with and without the gate, pipeline split on one and on two boards, horizontal Kafka scaling, and a Spark standalone cluster. It shows that decoupling the stages, not the filter alone or the second board, delivers most of the gain; (2) An end-to-end evaluation that follows every flow from send to verdict, reporting sustained capacity, latency percentiles with queueing included, energy per verdict, and the precision, recall and F1 of the deployed pipeline after gating, metrics that offline studies typically omit; (3) A same-model engine baseline (PySpark, NumPy, ONNX Runtime) that locates the cost of serving with Spark on the board; (4) Open, reproducible scripts integrated with a leakage-aware Spark ML training pipeline, so the deployed model is the one validated under a rigorous offline protocol. All code and configurations are publicly available[5].

2 Related Work

Edge and on-device intrusion detection. Edge computing pushes analytics closer to data sources to reduce latency and bandwidth [28, 11], and successive work has shrunk

[5] `https://github.com/vuthainguyen1602/Thesis_IDS` (branch `develop`).

IDS models until they fit single-board hardware. Early systems such as RPiDS [26] showed that a complete detection stack could run on a Raspberry Pi at all. Recent studies instead optimize the detector itself, via deep ensemble stacking on IoT-edge platforms [6], metaheuristic hyper-parameter search combined with explainability [21], and compact models tailored to industrial control traffic [31]. NVIDIA Jetson boards are a recurring target: autoencoder-based feature learning reaches sub-millisecond inference on a Jetson Nano [12], and generative models have been run on the Jetson Orin Nano to classify traffic and synthesize training data at once [8]. These studies share a single-device assumption: the detector is benchmarked in isolation on one board, so how detection work should be placed across several boards, and what a second board actually buys, remain open.

Streaming and distributed detection pipelines. Kafka [15] decouples producers from consumers and has become the de-facto ingest layer for streaming analytics, while Spark [30] and its MLlib library [19] supply the distributed training and scoring primitives. Combining the two for intrusion detection is an established pattern. Mohan Kiran Kumar et al. [20], for instance, route Kafka streams into Spark classifiers for distributed detection. Such pipelines are conceived for datacenter-class resources, where an additional executor is cheap, whereas on an 8 GB ARM board the JVM footprint is itself a design constraint, and the corresponding per-node CPU, memory, and energy traces are rarely published. More broadly, ML-based IDS surveys [14, 9, 29] catalog detection accuracy across dozens of models but seldom report the end-to-end throughput or latency percentiles of a running deployment.

Two-stage and split inference. Anomaly-detection surveys [5, 24] systematize the family of detectors that model normal behavior alone. This property is what makes a cascaded design attractive, in which a cheap unsupervised stage filters the bulk of normal data and a costlier supervised stage adjudicates the remainder. Unsupervised deep detectors, deep belief networks [1] and autoencoders [17, 12], are the usual choice for the cheap stage, since they can be calibrated on benign traffic alone. Distributing detection across devices has so far been explored mainly along the federated-learning axis, where nodes collaborate during training while inference stays local [23, 13]. Our split is orthogonal and concerns inference: the two stages are physically separated across boards and joined by a Kafka topic, so gate and classifier scale independently, and no prior work applies a PySpark MLlib classifier in this role on the NVIDIA Jetson Orin Nano Super.[6] Placement matters most for energy: recent TinyML work argues that energy per inference is a first-class edge metric [22], and skipping the heavy stage is the most direct way to lower it.

Evaluation protocol. A parallel line of work questions how faithfully offline scores transfer to deployment. Arp et al. [2] catalog recurring pitfalls in security ML, and Lanvin et al. [16] document labeling and duplication errors in CICIDS2017 [27] that measurably inflate reported performance. Sarhan et al. [25] similarly show that the choice of standard feature set shapes both generalizability and explainability. Our offline selection responds by choosing a leakage-free SHAP Top-30 model [18], and the present paper deploys exactly that model, so the operational numbers reported here do not be-

[6] `https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/`

long to a configuration that would fail a stricter offline audit. Concept drift [10] lies outside the present scope and motivates the retraining direction we outline in the conclusion. Table 1 positions our system against representative related work along the five axes above: multi-node role separation, streaming ingest, joint energy and latency reporting, leakage-aware model selection, and open code.

Table 1. Capability comparison with prior edge IDS deployments.

Reference	Platform	Multi-node roles	Streaming ingest	Energy & p95 latency	Leakage-aware model	Open code
Sforzin et al. [26]	Raspberry Pi	No	No	—	—	—
López-Martín et al. [17]	GPU server	No	No	—	—	—
Nguyen et al. [23]	IoT gateways	Federated	No	—	—	—
Hasan et al. [12]	1× Jetson Nano	No	No	—	—	—
Chithra Rani et al. [6]	1× Jetson Nano	No	No	—	—	—
Nasir et al. [21]	Edge-like (simulated)	No	No	—	—	—
The proposed system	**2× Jetson Orin Nano S.**	**Yes**	**Yes (Kafka)**	**Yes**	**Yes**	**Yes**

"—": not reported or not applicable.

3 System Architecture

3.1 Split Deployment Overview

Fig. 1 shows the split-deployment architecture. A host PC runs Kafka, PostgreSQL, InfluxDB, and Grafana. Two Jetson Orin Nano Super nodes consume messages from the network-flow topic. In pipeline-split mode, Jetson #1 runs the anomaly gate and forwards only suspicious flows to a dedicated suspicious-flow topic. Jetson #2 then applies the PySpark classifier and writes the classification results back to PostgreSQL. Both nodes stream performance metrics to InfluxDB/Grafana and raise alerts via Email/Slack (dashed arrows in Fig. 1).

3.2 Pipeline Roles

- **Anomaly gate:** a lightweight autoencoder + scaler over all 77 leak-free flow features. The MSE threshold is calibrated offline as a quantile (0.995) of a held-out benign validation split (10% of benign training flows, never the test set), so the operating point is committed before evaluation.
- **Classifier:** PySpark `PipelineModel` with the leakage-free SHAP-selected feature set.
- **Combined role:** gate and classifier in one process, used for horizontal scaling via Kafka consumer groups.

Each tier scores flows in batches of 20, or sooner once the oldest waiting flow has been held for 1 s, so the classifier, which receives only a small share of traffic, never holds a partial batch indefinitely.

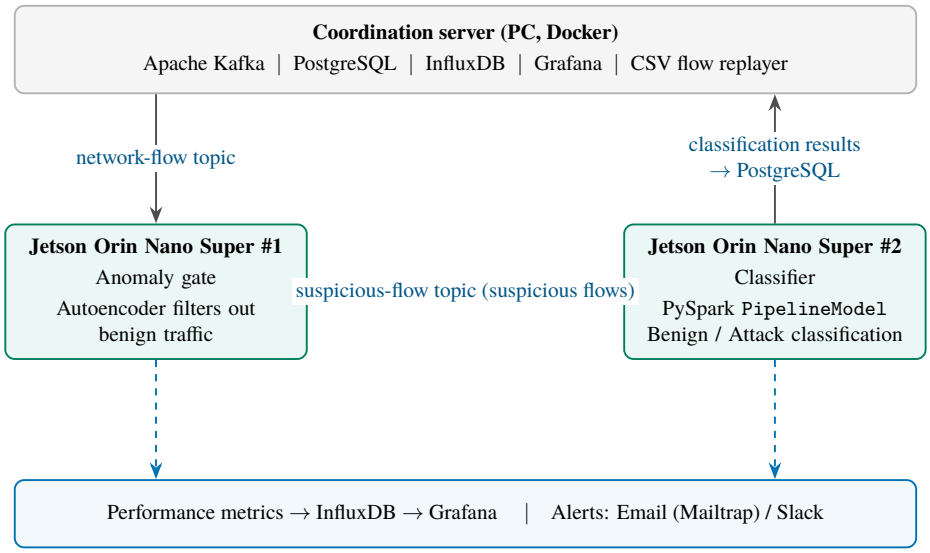


Fig. 1. Split-deployment architecture (PC + 2× Jetson Orin Nano Super).

3.3 Monitoring and Alerting

Each node tags its metrics with a unique node identifier in InfluxDB and PostgreSQL. Alerts are emitted from a designated primary node only, avoiding duplicate notifications. Fig. 2 shows the live Grafana dashboard while CICIDS2017 flows are replayed through the split deployment: prediction throughput, detected-attack count, inference latency, and per-node CPU/RAM utilization are all tracked in real time, alongside the most recent attack alerts persisted to PostgreSQL. Fig. 3 captures the dashboard at peak load in an earlier run with the previous model: the gate node idles at 4.6% CPU while the PySpark classifier node reaches 90.8%, so the classifier dominates the compute budget.

4 Implementation

Training and model export run on a Spark standalone cluster (host PC as master, two Jetson Orin Nano Super devices as workers) using Apache Spark [30] and its MLlib library [19]. The cluster topology is shown in Fig. 4. The host only coordinates (Spark master) and does not run an executor, so all reported training compute reflects the two homogeneous Jetson workers rather than the stronger host CPU. The edge stack is implemented in Python with configurable per-node roles. Every offline run logs its configuration (random seed, leakage/dedup switches, library versions) so each exported model is traceable to the exact pipeline that produced it.

5 From Spark Training to Edge Deployment

This paper does not compare Spark against Jetson Orin Nano Super as competing platforms. The two components address complementary phases of the same train–deploy



Fig. 2. Real-time Grafana monitoring of the edge IDS during CICIDS2017 replay.



Fig. 3. Grafana dashboard at peak load in the split deployment, showing the load asymmetry between the two pipeline roles.

pipeline (Fig. 5). Spark answers which model and how many features achieve acceptable accuracy on the full CICIDS2017 dataset [27]. The Jetson Orin Nano Super answers whether that model can then run in real time under memory, thermal, and bandwidth constraints at the network edge [28].

5.1 Offline Phase (Spark ML), Leakage-Aware

In the offline selection stage, we evaluate four feature configurations (baseline with all features, RF Importance Top-30, SHAP Top-30, and PCA with k=40) under a fixed 80/20 train–test split, using eight Spark ML classifiers and statistical validation (repeated paired resampling and permutation tests). PCA uses k=40 rather than 30 because it extracts components (linear combinations spreading variance across many axes) instead

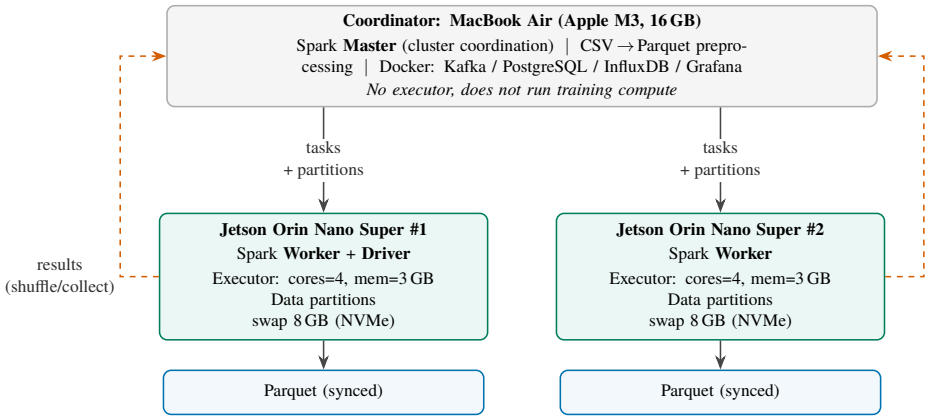


Fig. 4. Apache Spark standalone training cluster: the host as *master* (no executor), two Jetson Orin Nano Super devices as *worker/executor* nodes.

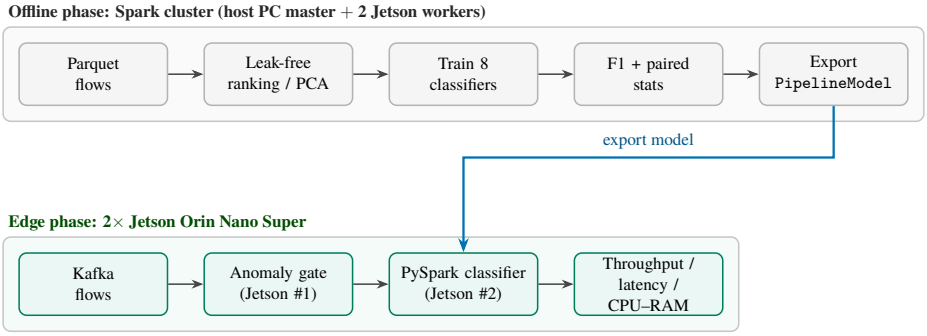


Fig. 5. Two-phase train–deploy pipeline linking Spark ML experiments to Jetson edge inference.

of selecting original features, so it needs more components to retain comparable variance. Across the common $\{20, 30, 40\}$ sweep, k=40 retains the most variance and is the fairest representative operating point for PCA. LightGBM is excluded because its SynapseML native libraries are x86_64-only and cannot run on the ARM64 Jetson Orin Nano Super deployment target. The offline protocol is leakage-aware [2]: it removes information that would be unavailable at deployment time but that otherwise inflates test scores. Motivated by documented labeling and duplication errors in CICIDS2017 [16], the label-leaking `destination_port` feature is removed and flows that are identical on all feature columns are de-duplicated before splitting. This yields realistic F1 values (Table 2) rather than the near-perfect scores obtained when these leaks are left in place. Offline metrics constrain which exported model is worth deploying. They are not directly comparable to edge throughput or latency.

Table 2. Bridge table: offline Spark selection criteria from the leakage-aware offline selection stage.

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	n/a[†]	Explainable; fewer features (deployed feature set)
RF Top-30 + XGBoost	0.9922	30	2.52	Same feature count; embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduction

[†]Not recoverable: the distributed save on the training cluster scattered this model's tree partitions across executors, so the recorded 3 KB covers only driver-side metadata. The deployed model is exported on a single node, where its size is measured reliably.

5.2 Edge Phase (Jetson Orin Nano Super)

The exported `PipelineModel` (VectorAssembler, StandardScaler, classifier) is loaded on Jetson #2 with only the selected SHAP features, reducing assembly and inference cost relative to the 77-feature baseline. The deployed input vector contains the 30 SHAP-selected features. The ranking is computed after `destination_port` has been removed as a label-leaking channel, consistent with the leakage-aware offline protocol. A lightweight autoencoder gate on Jetson #1 filters benign flows before they reach Spark inference, further lowering load on the classifier node.

We deploy the SHAP Top-30 feature set on Jetson #2 with a Random Forest classifier (the exported model scores test F1 0.9959, whereas the offline evaluation reports 0.9955 for this configuration, within 0.002 of the best XGBoost score in Table 2): it retains near-baseline F1 with substantially fewer features, and Random Forest is a native Spark ML estimator, so the exported `PipelineModel` loads on the ARM board without the additional native libraries that Spark's XGBoost integration requires. Sect. 7 then evaluates this choice under streaming load. The full four-method comparison and drift analysis are beyond the scope of this paper.

6 Experimental Setup

The testbed comprises two NVIDIA Jetson Orin Nano Super Developer Kits and a host PC running Docker Compose, connected over a Wi-Fi LAN. Each Jetson board pairs a 6-core Arm Cortex-A78AE CPU with an NVIDIA Ampere GPU featuring 1024 CUDA and 32 Tensor cores, delivering roughly 67 TOPS, and is equipped with 8 GB of LPDDR5 memory and a 256 GB NVMe SSD. On the software side, streaming inference runs on PySpark 3.4.1 over Apache Kafka 3.5 (Confluent Platform 7.5.0), while InfluxDB and Grafana collect and visualize runtime metrics. The workload replays 10,000 flows sampled from the CICIDS2017 test split (15.2% attacks, never seen in training). We compare a single node running the full pipeline, with and without the gate, against the three distributed modes A (pipeline split), B (horizontal Kafka scaling, full role on both boards) and C (Spark standalone cluster, host as master). A further placement, A1,

runs the pipeline split on one board, with gate and classifier as two separate processes on Jetson #2. A1 was measured in a later session with the same boards, models, replay file and network.

Every verdict stores the flow's producer timestamp and replay row, so each flow sent during a load window is followed to its verdict wherever it is issued. We report *sustained capacity*, the highest offered rate at which every flow sent in a 45 s window receives its verdict within 10 s of the window's end (about one PySpark batch); send-to-verdict latency p50/p95 over those flows, queueing included (the offset between the host and board clocks is measured before and after every window and subtracted); the gate-skip ratio and the confusion counts of the final verdicts computed from the flows themselves; and energy per verdict from `tegrastats` at the sustained rate. Each rate is run twice after a warm-up, with a fresh Kafka consumer group per run so no run inherits another's backlog. Because every window replays the sample from its first row, the capacity windows cover only its first few thousand flows. For the detection metrics, one additional 500 s run of mode A at 20 flows/s, below its sustained rate, replays all 10,000 flows once.

7 Results

A host-side script starts the pipeline roles on the Jetsons over SSH, drives the load and collects the per-flow records. A sweep raises the offered rate per mode until flows stop finishing within the 10 s bound. Table 3 and Fig. 6 summarize the sustained operating point of each mode.

Table 3. Sustained operating point per deployment mode on the 2-node Jetson cluster: the highest rate at which every flow got its verdict within 10 s of the load window, with latency measured from send to verdict.

Mode	Boards	Sustained (flows/s)	e2e p50 / p95 (s)	Gate skip (%)	J / verdict
Single node, gate off	1	2.5	5.9 / 11.3	–	3.04
Single node + gate	1	1.9*	3.8 / 7.8	89.0	3.04
B: Horizontal, gate off	2	3.3	4.1 / 8.2	–	3.20
C: Spark cluster	2	$\geq$8.7†	0.7 / 5.2	88.6	1.15
A1: Split, one board	1	17.8‡	0.6 / 5.3	88.9	0.32
A: Pipeline split	2	22.4	1.0 / 7.1	88.8	0.60

Energy is total-board power over verdict rate, measured at 2.0, 2.0, 3.9, 9.5, 17.8 and 18.4 flows/s respectively; both boards are summed in two-node modes. Gate skip is pooled over all runs of a mode. Detection metrics are in Table 5. ‡Delivered at an offered 20 flows/s; at 30 flows/s one repeat drained in 16.1 s. *At 3 flows/s the single node with and without the gate both deliver 2.4–2.5 verdicts/s; the gated one falls to the 2 flows/s step only because one of its two repeats drained in 10.6 s. †Lower bound: at 20 flows/s one repeat kept up at 17.1 flows/s and the other drained in 12.3 s.

Capacity. Pipeline split sustains 22.4 flows/s, about 9$\times$ the single node that runs the whole pipeline (2.5 flows/s). The gate alone does not explain the gain: with the gate

added to the single node, where gate and classifier run one after the other in the same process, it shows no measurable gain (2.4–2.5 verdicts/s at 3 flows/s, the same as without the gate), because each batch still waits for a PySpark call that costs seconds regardless of how few flows reach it. Pipeline split decouples the two stages, so the gate keeps consuming while the classifier works through its share. Decoupling, not the second board, accounts for most of the gain: the same split as two processes on one board (A1) sustains 17.8 flows/s, 7× the single node, and the second board adds about a quarter on top. Horizontal scaling of the full role (gate off) stays Spark-bound at 3.3 flows/s. The Spark standalone cluster sustains at least 8.7 flows/s. At 20 flows/s one of two repeats kept up (17.1 flows/s) and the other did not, so its capacity lies between about 9 and 17 flows/s, below pipeline split and with an extra host dependency.

Latency. At its sustained rate, pipeline split returns a median verdict in about 1 s, dominated by gate-only verdicts. Flows forwarded to the classifier wait for a PySpark batch, and the p95 over all flows is 7.1 s. Every mode shows a multi-second tail for the same reason: one PySpark batch on the board takes seconds regardless of its size.

Energy efficiency. Power is sampled with `tegrastats` (30 s idle baseline, then the load window). Serving 18.4 flows/s on two boards, pipeline split spends 0.60 J per verdict against 3.04 J for a single node at its 2 flows/s, a $5\times$ reduction. On one board, A1 needs only 0.32 J per verdict at 17.8 flows/s, since the second board's idle draw disappears. The active (idle-subtracted) power rises by about 1.5 W under load in every mode.

Inference-engine baseline (cost of the unified stack). We deploy a Spark `PipelineModel` on the edge for consistency with the distributed training phase: the same stack trains on the cluster and serves on the device. To quantify the price of that unification, we export the deployed forest to NumPy and ONNX Runtime and serve all three through the edge's own inference layer on the classifier node, batch size 10 (Table 4). The three engines return identical predictions on every replayed flow. ONNX Runtime serves the same forest at about 10,100 flows/s, about $7{,}200\times$ the PySpark path, at $<0.01\%$ of its active energy per inference. The gap motivates exporting the `PipelineModel` to ONNX/TensorRT for serving while retaining Spark for training. Wiring the ONNX session into the streaming pipeline is left as future work. A batch-size sweep on the same board locates the PySpark cost: a call takes 7.1–7.4 s from batch 10 to 1,000 and 9.5 s at 10,000, so the cost is a fixed per-call overhead, not the trees. PySpark throughput therefore grows almost in proportion to batch size (1,046 flows/s at 10,000) while ONNX Runtime saturates near 28,000 flows/s, and the gap narrows to about $27\times$. A streaming IDS cannot batch its way out: at 20 flows/s, collecting 10,000 flows takes over 8 minutes.

Gate trade-off. The anomaly gate is tunable rather than a single fixed point: sweeping its threshold trades attack recall against the fraction of flows offloaded from Spark (Fig. 7). At the pre-committed quantile of 0.995 it forwards 68.1% of attack flows at a 0.49% false-positive rate while skipping 89.4% of all flows offline. Flows the gate skips are final, so the gate bounds the system's recall. Scoring all 77 features matters: restricted to the classifier's 30 SHAP features, the same gate forwards only 44.8% of attack flows at the same quantile, because a subset chosen to separate the classes is not the one that exposes anomalies.

Table 4. Single-node inference cost of the deployed forest (same trees, identical predictions) across engines, batch size 10.

Engine	Throughput (flows/s)	Batch p95 (ms)	Active energy/inf. (mJ)
PySpark (deployed)	1.4	7515	1506
NumPy	54.8	194	11.3
ONNX Runtime	10,092	1.2	0.087

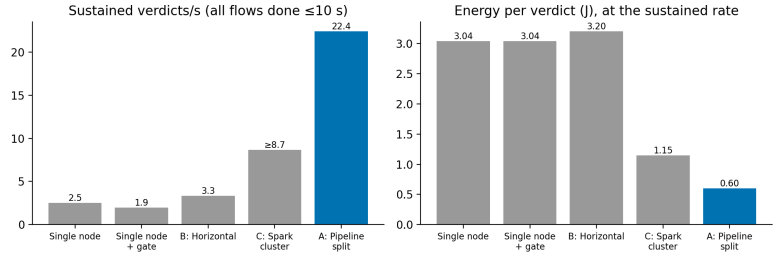


Fig. 6. Sustained verdict throughput (left) and energy per verdict at that rate (right) per deployment mode. The Spark-cluster bar is a lower bound (Table 3).

End-to-end detection after gating. Table 5 scores the final verdicts of the deployed pipeline against the replay labels, with gate-only verdicts counted as benign. In the run that replays all 10,000 flows once (mode A at 20 flows/s, sustained for 500 s with a 5.9 s drain and an end-to-end p95 of 5.1 s), the pipeline detects 1,055 of 1,518 attack flows (recall 69.5%) and raises no false alarm on 8,483 benign flows, so F1 is 0.82, against 0.996 offline for the classifier alone. All 463 missed attacks were skipped by the gate. The classifier made no error on the flows it received. Every placement with the gate lands at the same point (recall 68–73%, no false positive), and without the gate the deployed pipeline makes no error on the windows replayed. Gating thus turns the classifier's offline F1 into an end-to-end F1 that the gate's threshold alone decides.

Table 5. End-to-end detection of the deployed pipeline: final verdicts against replay labels, gate-only verdicts counted as benign.

Mode	Verdicts	Attack	FN gate / clf	FP	Recall (%)	F1
A: full replay (10,000 flows)	**10,001**	**1,518**	**463 / 0**	**0**	**69.5**	**0.820**
A: capacity windows	19,543	2,990	898 / 0	0	70.0	0.823
A1: split, one board	5,858	882	278 / 0	0	68.5	0.813
C: Spark cluster	2,499	381	115 / 0	0	69.8	0.822
Single node + gate	1,480	200	55 / 0	0	72.5	0.841
Gate off (single, B)	1,855	242	– / 0	0	100.0	1.000

Precision is 100% in every row (no benign flow received an attack verdict). Capacity-window rows pool every repeat of a mode. Each window replays the sample from its first row, so these rows overlap and cover 3,601 (A), 1,351 (A1), 829 (C) and 219 (single node + gate) distinct flows. The full-replay row covers all 10,000 once (the sender's loop repeats one row).

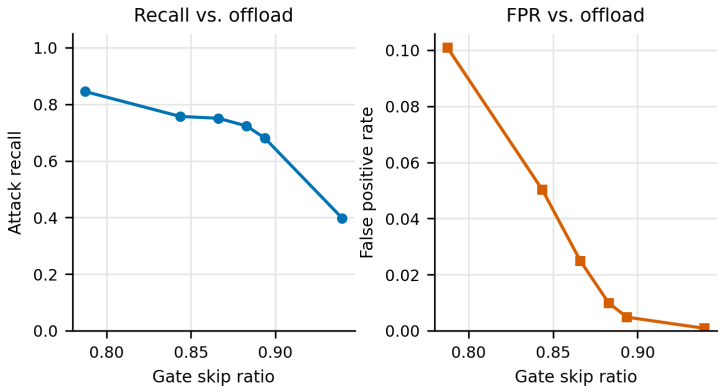


Fig. 7. Anomaly-gate operating curve: attack recall (left) and false-positive rate (right) vs. gate-skip ratio.

8 Discussion

Pipeline split relieves the Spark classifier of about 89% of the traffic, and that is what lifts sustained capacity from 2.5 to 22.4 flows/s. Two findings qualify the design. First, decoupling the stages matters more than the filter or the hardware: the same gate run in the classifier's process gains nothing, two decoupled processes on one board reach 17.8 flows/s, and a second board adds about a quarter more at roughly twice the energy per verdict. With one board available, the one-board split is the more efficient choice. A second board buys headroom. Second, the throughput is paid for in recall: at the pre-committed threshold about 30% of attack flows are skipped as benign, and Table 5 shows that these skips are the pipeline's only errors. An operator trading recall for capacity should therefore do so knowingly along the curve of Fig. 7.

Dependence on the host. The boards run all detection, but Kafka, PostgreSQL, InfluxDB and Grafana run on the host PC, so the deployment is not autonomous. Flows enter the pipeline through the host's broker and verdicts are stored there. If the host fails, detection stops, even though both boards remain healthy. With the PySpark classifier the host is not the bottleneck: the boards saturate first, and a round trip from host to board averages 4–9 ms on the Wi-Fi LAN. The coupling still bounds scaling. Every added board adds one more consumer of the same broker and one more writer to the same database, and in our setup each verdict is a separate database insert. A field deployment would therefore move ingest to the edge: a broker on, or next to, the board that receives the flow export, with verdicts buffered locally and forwarded to a central store in batches, so that detection keeps running when the uplink or the host is lost. Kafka's replication would let such a broker be spread over the boards themselves. We have not measured these variants. They trade the memory the JVM-based broker takes on an 8 GB board against independence from the host.

Beyond CICIDS2017. Every number above comes from CICIDS2017 replay. To test how far the offline selection transfers, we trained the same model family (a class-weighted Random Forest under the same leakage-aware protocol, on the flow features both datasets share) on CICIDS2017 and tested it on a 20% stratified sample of CSE-CIC-IDS2018 [3], and vice versa, over 18 runs. In-domain F1 is 0.995 and 0.925, but across datasets AUC-PR falls to 0.48 (2017→2018) and 0.19 (2018→2017), and no cross-dataset run exceeds F1 0.31, mostly through missed attacks. This matches earlier inter-dataset studies [7, 4]. Training on 1% of the target dataset's labelled flows alone restores F1 to 0.92 and 0.99. The capacity, latency and energy results depend on the pipeline rather than the data. The detection results do not carry over to another network without retraining, and the gate's threshold would have to be recalibrated on that network's benign traffic.

Limitations. Benchmarks are obtained on a two-node lab cluster over Wi-Fi. Wired networking and larger clusters may shift latency tails. Capacity is bounded by the PySpark classifier, whose per-batch cost of several seconds sets both the tail latency and the ceiling. The ∼67-TOPS GPU remains unexploited by the CPU-only pipeline. The gate bounds recall, and its threshold is calibrated on benign training data only. Live-traffic re-calibration is future work. Energy figures subtract a single pre-run idle baseline, so within-run drift in background power is not captured. Detection results are tied to CICIDS2017 replay and do not transfer across datasets without retraining (see above). Live traffic and deliberately evasive (adversarial) traffic remain untested, and the host-side services were not moved to the edge.

9 Conclusion

We presented a reproducible distributed edge IDS for Jetson Orin Nano Super clusters that pairs Kafka streaming with PySpark inference and deploys a model validated under a leakage-aware offline protocol. Measured from send to verdict with queueing included, placing a lightweight autoencoder gate and the Spark classifier on separate boards sustains 22.4 flows/s against 2.5 for a single node running the whole pipeline (≈9×), at an end-to-end p95 of about 7 s and 0.60 J per verdict against 3.04 J. The gain requires decoupling the stages: the same gate run in the classifier's process brings no measurable gain, whereas the two stages as separate processes on one board already sustain 17.8 flows/s at 0.32 J per verdict. It is paid for in recall. Over all 10,000 replayed flows the pipeline raises no false alarm but detects 69.5% of attacks (F1 0.82), and every miss is a flow the gate skipped. Horizontal scaling of the full role stays Spark-bound (3.3 flows/s) and the Spark standalone cluster sustains between about 9 and 17 flows/s, so pipeline split remains our recommended default. Serving the same forest, ONNX Runtime is about 7,200× faster than the deployed PySpark path at its batch size of 10, with identical predictions, a concrete target for the next iteration of the serving layer. Future work includes TensorRT optimization, live-traffic validation, and federated retraining under drift [13].

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Alrawashdeh, K., Purdy, C.: Toward an online anomaly intrusion detection system based on deep learning. In: Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA). pp. 195–200 (2016). https://doi.org/10.1109/ICMLA.2016.0040
2. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
3. Canadian Institute for Cybersecurity and Communications Security Establishment: CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC). `https://www.unb.ca/cic/datasets/ids-2018.html` (2018)
4. Cantone, M., Marrocco, C., Bria, A.: Machine learning in network intrusion detection: A cross-dataset generalization study. IEEE Access **12**, 144489–144508 (2024). https://doi.org/10.1109/ACCESS.2024.3472907
5. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3), 1–58 (2009). https://doi.org/10.1145/1541880.1541882
6. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. Discover Applied Sciences **7**, 928 (2025). https://doi.org/10.1007/s42452-025-06871-z
7. D'hooge, L., Wauters, T., Volckaert, B., De Turck, F.: Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. Journal of Information Security and Applications **54**, 102564 (2020)
8. Diaz-Gorrin, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. Applied Sciences **16**(3), 1442 (2026). https://doi.org/10.3390/app16031442
9. Ferrag, M.A., Maglaras, L., Moschoyiannis, S., Janicke, H.: Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. Journal of Information Security and Applications **50**, 102419 (2020)
10. Gama, J., Zliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 1–37 (2014)
11. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. Sensors **20**(22), 6441 (2020). https://doi.org/10.3390/s20226441
12. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
13. Hernandez-Ramos, J.L., Karopoulos, G., Chatzoglou, E., Kouliaridis, V., Marmol, E., Gonzalez-Vidal, A., Kambourakis, G.: Intrusion detection based on federated learning: A systematic review. ACM Computing Surveys **57**(12), 1–65 (2025). https://doi.org/10.1145/3731596
14. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity **2**(1), 1–22 (2019)
15. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)
16. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In: Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers. pp. 18–33. Lecture Notes in Computer Science, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2

17. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT. Sensors **17**(9), 1967 (2017). https://doi.org/10.3390/s17091967
18. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems **30** (2017)
19. Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M.J., Zadeh, R., Zaharia, M., Talwalkar, A.: MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research **17**(34), 1–7 (2016)
20. Mohan Kiran Kumar, K., Sreekar, M., Ullas, K., Supriya, M.: Distributed intrusion detection system using Kafka and Spark streaming. In: 2025 International Conference on Visual Analytics and Data Visualization (ICVADV). pp. 302–309. IEEE (2025). https://doi.org/10.1109/ICVADV63329.2025.10961691
21. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozae, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. Scientific Reports **16**, 2313 (2026). https://doi.org/10.1038/s41598-025-32183-8
22. Nassef, L., Alghamdi, M.I., Ben Chaabane, S., Abbas, Q., Alawad, W.M., Albalawi, O.H., Alqaisi, O.I., Fakieh, B.: Lightweight and energy-aware intrusion detection for industrial IoT using TinyML and edge AI. Scientific Reports **16**(1), 18524 (2026). https://doi.org/10.1038/s41598-026-50690-0
23. Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: Proc. IEEE 39th Int. Conf. on Distributed Computing Systems (ICDCS). pp. 756–767 (2019). https://doi.org/10.1109/ICDCS.2019.00080
24. Ruff, L., Kauffmann, J.R., Vandermeulen, R.A., Montavon, G., Samek, W., Kloft, M., Dietterich, T.G., Müller, K.R.: A unifying review of deep and shallow anomaly detection. Proceedings of the IEEE **109**(5), 756–795 (2021)
25. Sarhan, M., Layeghy, S., Portmann, M.: Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection. Big Data Research **30**, 100359 (2022). https://doi.org/10.1016/j.bdr.2022.100359
26. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom). pp. 440–448 (2016). https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCom-IoP-SmartWorld.2016.0080
27. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116 (2018)
28. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal **3**(5), 637–646 (2016). https://doi.org/10.1109/JIOT.2016.2579198
29. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep learning approach for intelligent intrusion detection system. IEEE Access **7**, 41525–41550 (2019)
30. Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S., Stoica, I.: Spark: Cluster computing with working sets. In: Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10) (2010)
31. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. Electronics **15**(3), 644 (2026). https://doi.org/10.3390/electronics15030644